

# UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

## FACULTAD DE INGENIERÍA



**Lab Organización y Arquitectura de Computadoras**
**Grupo: 03**

# Practica 2


**P R E S E N T A**

Rea Alberto Cristian | 318273130
Ortega de la Paz Rafael | 420054085

**PROFESOR DE LABORATORIO**

Ricardo Daniel Vega Gutierrez


Fecha de entrega: 6 de Octubre de 2026
**Semestre 2027-1**



## Objetivo(s)

Analizar, diseñar e implementar una máquina de estados finitos a partir de una carta ASM, identificando el funcionamiento de los flip-flops tipo D como elementos de memoria. Obtener las funciones de estado siguiente y salida mediante el llenado de memoria y mapas de Karnaugh, para posteriormente comprobar su funcionamiento utilizando la FPGA DE10-Lite.

## Introducción

Una máquina de estados finitos (FSM, por sus siglas en inglés) es un modelo de circuito secuencial cuyo comportamiento depende tanto de las entradas actuales como del estado en que se encuentra, estado que queda almacenado en elementos de memoria — en este caso, flip-flops tipo D. A diferencia de un circuito puramente combinacional, una FSM "recuerda" su historia reciente a través de ese estado, lo que le permite reaccionar de forma distinta ante la misma entrada según el punto de la secuencia en que se encuentre.

La carta ASM (Algorithmic State Machine) es una herramienta gráfica para especificar el comportamiento de una FSM: cada estado se representa como un bloque rectangular con sus salidas asociadas, y las condiciones de transición entre estados se representan mediante bloques de decisión. A partir de esta carta es posible construir la tabla de transiciones y, mediante mapas de Karnaugh, obtener las ecuaciones booleanas simplificadas tanto para el estado siguiente como para las salidas — ecuaciones que después se implementan en hardware con compuertas lógicas y flip-flops tipo D, y se verifican experimentalmente programando el diseño en una FPGA.

## Desarrollo

### 1. Estados, entradas y salidas

Al revisar la carta ASM de la práctica (Figura 1) contamos cuatro estados, que llamamos EST0, EST1, EST2 y EST3. Hay dos entradas, **X** y **Y**, que aparecen en los rombos de decisión, y dos salidas, **S1** y **S0**. Las salidas están dentro de los rectángulos de estado, es decir que solo dependen del estado en el que está la máquina y no de X ni de Y (máquina tipo Moore). Lo que valen las salidas en cada estado es:

| Estado | Salidas activas | S1 | S0 |
|---|---|---|---|
| EST0 | S1 | 1 | 0 |
| EST1 | S0 | 0 | 1 |
| EST2 | ninguna | 0 | 0 |
| EST3 | S1 y S0 | 1 | 1 |

Para leer la carta: en EST0 la máquina se queda ahí si X = 0, y si X = 1 pregunta por Y; con Y = 0 regresa a EST0 y con Y = 1 pasa a EST1. En EST1 se queda mientras Y = 0, y si Y = 1 revisa X: con X = 0 sigue en EST1 y solo cuando X = 1 pasa a EST2. De EST2 siempre va a EST3, y de EST3 siempre regresa a EST0. O sea que los dos caminos que salen de EST0 y EST1 hacia adelante necesitan que **X y Y valgan 1 al mismo tiempo**.

## 2. Carta ASM

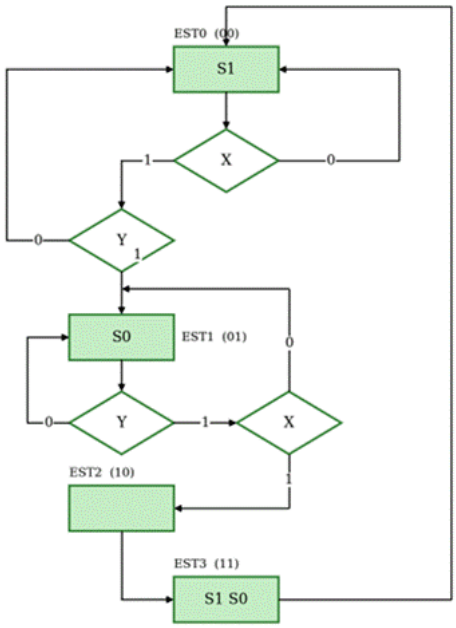



*Figura 1. Carta ASM de la práctica redibujada, con la codificación de cada estado entre paréntesis.*

3. Codificación de estados

Con cuatro estados bastan 2 bits ($2^2 = 4$), por lo que se usan dos flip-flops tipo D, Q1 y Q0. Usamos la codificación binaria en orden, que es la más directa:

| Estado | Q1 | Q0 |
|---|---|---|
| EST0 | 0 | 0 |
| EST1 | 0 | 1 |
| EST2 | 1 | 0 |
| EST3 | 1 | 1 |

Como el flip-flop D copia en Q lo que tiene en D en el siguiente pulso de reloj, las entradas D1 y D0 son directamente el estado siguiente, $Q1^+$ y $Q0^+$. Por eso en la tabla de transiciones el estado siguiente es lo mismo que D1 D0.

### 4. Tabla de transiciones

La tabla tiene 16 renglones (2 bits de estado y 2 entradas). Cada renglón se obtuvo siguiendo las flechas de la carta desde el estado y con los valores de X y Y que corresponden. Las salidas se anotan según el estado actual.

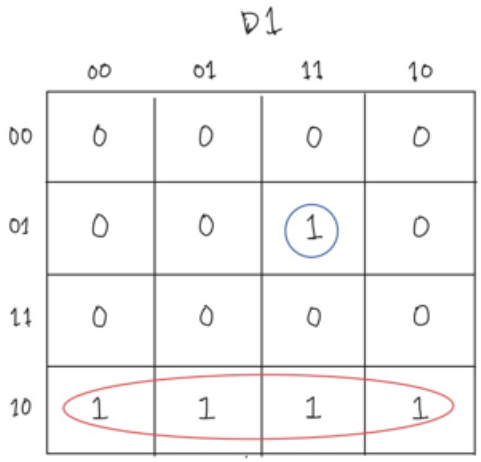
| Q1 | Q0 | X | Y | Estado actual | Q1+ (D1) | Q0+ (D0) | Estado sig. | S1 | S0 |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | EST0 | 0 | 0 | EST0 | 1 | 0 |
| 0 | 0 | 0 | 1 | EST0 | 0 | 0 | EST0 | 1 | 0 |
| 0 | 0 | 1 | 0 | EST0 | 0 | 0 | EST0 | 1 | 0 |
| 0 | 0 | 1 | 1 | EST0 | 0 | 1 | EST1 | 1 | 0 |
| 0 | 1 | 0 | 0 | EST1 | 0 | 1 | EST1 | 0 | 1 |
| 0 | 1 | 0 | 1 | EST1 | 0 | 1 | EST1 | 0 | 1 |
| 0 | 1 | 1 | 0 | EST1 | 0 | 1 | EST1 | 0 | 1 |
| 0 | 1 | 1 | 1 | EST1 | 1 | 0 | EST2 | 0 | 1 |
| 1 | 0 | 0 | 0 | EST2 | 1 | 1 | EST3 | 0 | 0 |
| 1 | 0 | 0 | 1 | EST2 | 1 | 1 | EST3 | 0 | 0 |
| 1 | 0 | 1 | 0 | EST2 | 1 | 1 | EST3 | 0 | 0 |
| 1 | 0 | 1 | 1 | EST2 | 1 | 1 | EST3 | 0 | 0 |
| 1 | 1 | 0 | 0 | EST3 | 0 | 0 | EST0 | 1 | 1 |
| 1 | 1 | 0 | 1 | EST3 | 0 | 0 | EST0 | 1 | 1 |
| 1 | 1 | 1 | 0 | EST3 | 0 | 0 | EST0 | 1 | 1 |
| 1 | 1 | 1 | 1 | EST3 | 0 | 0 | EST0 | 1 | 1 |

*Tabla 1. Tabla de transiciones de la máquina.*

### 5. Mapas de Karnaugh

Hicimos un mapa para cada entrada de los flip-flops (D1 y D0) y uno para cada salida (S1 y S0). En los mapas de D1 y D0 las filas son Q1Q0 y las columnas XY, ambas en código Gray (00, 01, 11, 10) para que las celdas vecinas cambien de un solo bit. El número chico en cada celda es el número de minitérmino de Q1Q0XY. Para escribir las ecuaciones usamos el apóstrofo como negación (Q0' = no Q0).

**Función D1.** Vale 1 en los minitérminos 7, 8, 9, 10 y 11. La fila Q1Q0 = 10 completa (EST2, que siempre avanza a EST3) forma un grupo de 4. El minitérmino 7 (EST1 con X = 1 y Y = 1) no tiene vecinos con 1, así que queda solo y se escribe completo.



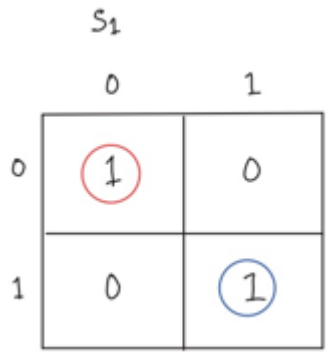

D1 = Q1 Q0' + Q1' Q0 X Y

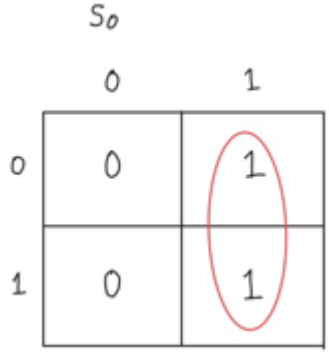
**Función D0.** Vale 1 en los minitérminos 3, 4, 5, 6, 8, 9, 10 y 11. Sacamos cuatro grupos: la fila 10 completa (Q1 Q0'); los minitérminos 4 y 5 (Q1' Q0 X'); los minitérminos 4 y 6, que son vecinos porque las columnas 00 y 10 se tocan por los bordes del mapa (Q1' Q0 Y'); y los minitérminos 3 y 11, que son vecinos porque las filas 00 y 10 también se tocan por los bordes (Q0' X Y).




D0 = Q1 Q0' + Q1' Q0 X' + Q1' Q0 Y' + Q0' X Y

**Salidas S1 y S0.** Como son salidas tipo Moore no dependen de X ni de Y, por lo que los mapas son solo de Q1 y Q0. S1 vale 1 en EST0 (00) y en EST3 (11), que están en esquinas opuestas y no se pueden agrupar; S0 vale 1 siempre que Q0 = 1 (EST1 y EST3), y esos dos unos forman un grupo de 2.




S1 = Q1' Q0' + Q1 Q0

S0 = Q0

### 6. Ecuaciones finales

Con lo anterior, las funciones que implementamos son:

$$D1 = Q1\ Q0' + Q1'\ Q0\ X\ Y$$

$$D0 = Q1\ Q0' + Q1'\ Q0\ X' + Q1'\ Q0\ Y' + Q0'\ X\ Y$$

$$S1 = Q1'\ Q0' + Q1\ Q0$$

$$S0 = Q0$$

Para comprobarlas revisamos algunos renglones de la tabla. Por ejemplo, con Q1Q0 = 01 (EST1), X = 1 y Y = 1 la ecuación de D1 da 1 (por el término Q1' Q0 X Y) y la de D0 da 0 (ningún término se cumple), entonces el estado siguiente es 10 = EST2, que es lo que dice la carta. Con Q1Q0 = 01, X = 0 y Y = 1, D1 da 0 y D0 da 1 (por Q1' Q0 X'), así que se queda en 01 = EST1, que también coincide.

**Conclusiones**

**Ortega de la Paz Rafael**

Con esta práctica entendí mejor cómo se pasa de una idea de funcionamiento a un circuito concreto. Partimos de una carta ASM con cuatro estados y dos entradas, y el trabajo consistió en ir traduciéndola paso a paso: primero los estados y las salidas, luego la codificación en dos bits, después la tabla de transiciones y al final las ecuaciones de cada flip-flop. Ver que cada paso sale del anterior me ayudó a no sentir el diseño como algo abstracto.

Lo que más me sirvió fue la tabla de transiciones, porque ahí se vio claro que la máquina solo avanza de EST0 a EST1 y de EST1 a EST2 cuando X y Y valen 1 a la vez, y que en cualquier otro caso se queda o regresa. También noté que usar flip-flops tipo D simplifica el diseño, ya que la entrada D es directamente el estado siguiente y no hay que hacer otra tabla de excitación como pasaría con otro tipo de flip-flop.

En los mapas de Karnaugh me di cuenta de que conviene revisar los bordes: en D0 hay grupos que parecen separados pero en realidad son vecinos, y si no se aprovechan la ecuación queda más larga. Las salidas fueron más sencillas porque dependen solo del estado, y S0 terminó siendo igual a Q0. Para verificar el resultado probé algunos renglones de la tabla en las ecuaciones y coincidían con la carta.

Considero que la práctica cumplió su objetivo, porque dejó claro que un buen análisis previo en papel reduce errores cuando después se implementa y se prueba en la FPGA.

**Rea Alberto Cristian**

En esta práctica pasé una carta ASM a ecuaciones que se pueden armar con flip-flops tipo D y compuertas. Lo primero fue leer bien la carta: al principio las flechas de regreso de los rombos se me hacían confusas, pero después de seguir cada camino vi que las dos condiciones que hacen avanzar a la máquina, en EST0 y en EST1, son X = 1 y Y = 1 al mismo tiempo, y que cualquier otra combinación la deja en el mismo estado o la regresa a EST0. Con eso la tabla de transiciones salió de forma directa.

Los mapas de Karnaugh fueron la parte donde más cuidado hubo que tener. En D0 algunos grupos no se ven pegados en el dibujo y solo son vecinos porque el mapa se cierra por los bordes (los minitérminos 4 y 6, y el 3 y el 11); si no se toman, la ecuación queda con más términos de los necesarios. También me quedó claro que las salidas S1 y S0 dependen solo del estado, por eso sus ecuaciones son tan cortas (S0 = Q0) comparadas con las de estado siguiente. Comprobé algunos renglones de la tabla sustituyendo en las ecuaciones y coincidieron con la carta, lo cual me dio confianza de que la simplificación estaba bien antes de pasarla a VHDL y a la tarjeta DE10-Lite.

Me quedo con que hacer primero el análisis en papel ayuda mucho a la hora de programar, porque ya se sabe qué debe hacer cada estado y qué resultado esperar en cada transición, y eso sirve para detectar errores cuando se prueba el circuito en la FPGA.

Bibliografía


Vega Gutiérrez, R. D. (2026). Notas del curso Organización y Arquitectura de Computadoras. Facultad de Ingeniería, UNAM.

Mano, M. M. y Ciletti, M. D. (2018). Digital Design (6.ª ed.). Pearson.

Intel Corporation. (2021). Intel Quartus Prime Lite Edition User Guide.

Terasic Technologies. (2017). DE10-Lite User Manual.